CS 578: CYBER-SECURITY

PART II: OS SECURITY II

Sanghyun Hong
sanghyun.hong@oregonstate.edu





RECAP – WHERE WE ARE

- Part I – Network/Internet security
  - The attacker is *outside* your machine
- Part II – Computer systems security
  - The attacker is *inside* your machine
  - Memory safety: how the attacker gets inside your machine (Buffer overflow)
  - OS security I: once inside, how can we *reduce* what the attacker can do?
    - SetUID demystified – drop the privileges that you don't need
    - Capsicum – Never hold those privileges in the first place

CAPSICUM – COMPARTMENTALIZATION

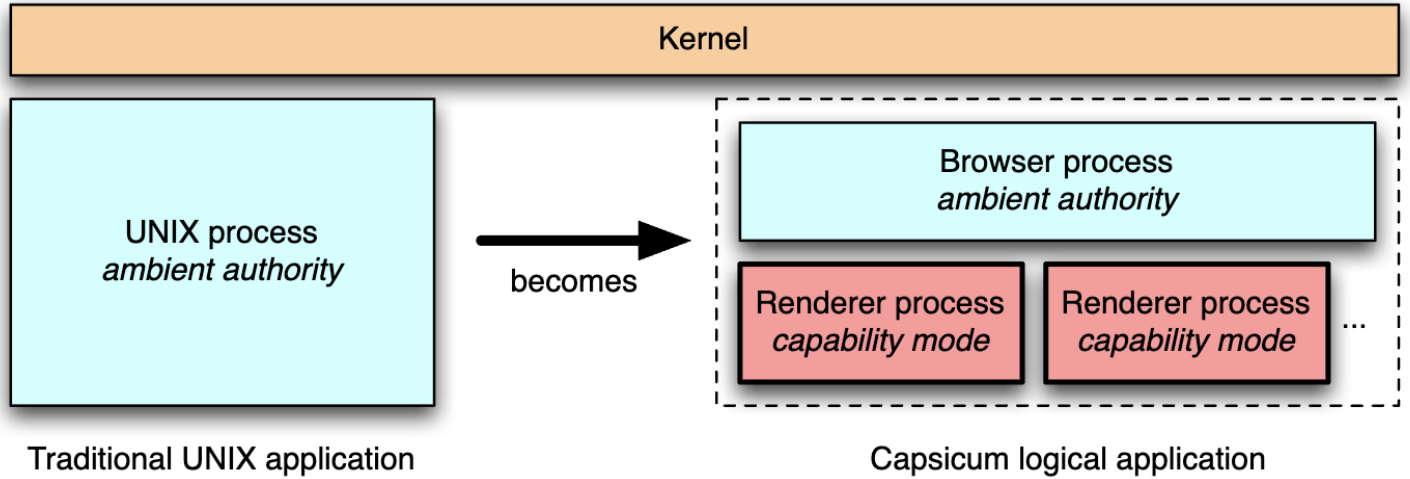
THE LIMITATION OF SETUID

- Suppose that you decompress a program with gzip
  - Gzip unfortunately contains a buffer overflow vulnerability
  - setuid makes sure that the attacker cannot access /etc/passwd
  - But…

- Your ~/.ssh/<ssh_key>
  - The attacker can read ~/.ssh/id_rsa and send it anywhere
  - The code runs with *your UID*
  - setuid protects your program from Alice
  - But it cannot from your own program processing untrusted input

CAPSICUM

- Key idea: compartmentalization



  - Split one program into pieces with different privileges
  - Privileged part (opens files, sockets) vs. Sandboxed part (parses the dangerous input)

SO FAR, OS IS NOT DESIGNED TO SUPPORT THIS

Chromium: one browser, six different sandbox implementations

OS	Mechanism	Lines of code
Windows	ACLs + SIDs	22,350
Linux	chroot (setuid-root helper)	605
Mac OS X	Seatbelt (MAC)	560
Linux	SELinux (MAC)	200
Linux	seccomp + userspace wrapper	11,301
FreeBSD	Capsicum	100

*Lines of code = how little the OS helped; not how safe it is

How to implement the compartmentalization

- Opening a file needs two steps:
  - Use a system call with names, e.g., open("/home/<usr>/.ssh/<your-key>")
  - OS then checks who asks – "Is this process's user allowed to do it?"

- Capsicum: no names, no "who," only the keys

BEFORE		AFTER GRANTED	
open("/etc/passwd")	O	open("/etc/passwd")	X
read(fd_input)	O	read(fd_input)	O

- Two kernel additions
  - cap_enter() do not use name-based approaches
  - Capabilities do use keys that grant what they can open

CAPSICUM – CAPABILITY MODE

- After cap_enter(), anything that needs a NAME fails
  - open("/etc/passwd") X file paths
  - kill(123, SIGKILL) X process IDs
  - connect("8.8.8.8:53") X network addresses

- Anything that a process holds keep working:
  - read(fd_input) O
  - write(fd_output) O

- One-way ticket
  - setuid can keep a way back (root is in the saved UID)
  - Capsicum has no way back at all; no system call to leave

CAPSICUM – CAPABILITIES

- Capabilities = file descriptors + rights
  - cap_new(fd, rights) create a key (fd) that can do only the “rights”
  - pdfork fork that hands the key to the child, not a PID

```
// ① BEFORE the lock — names still work. Grab what you will need
int lib = open("/lib", O_RDONLY | O_DIRECTORY);   // a DIRECTORY key
int fd  = open("photo.jpg", O_RDWR);              // read + write
int ro  = cap_new(fd, CAP_READ | CAP_SEEK);       // weaker copy    ✓

// ② LOCK
cap_enter();

// ③ AFTER the lock — names are gone, keys still work
open("/lib/libc.so.7", O_RDONLY);                 // a name         ✗
openat(lib, "libc.so.7", O_RDONLY);               // under /lib     ✓
openat(lib, "../etc/passwd", O_RDONLY);           // climbs out     ✗
read(ro, buf, n);                                 // has READ       ✓
write(ro, buf, n);                                // no WRITE       ✗
cap_new(ro, CAP_READ | CAP_WRITE);                // add a right?   ✗
```

CAPSICUM – TCPDUMP IN TWO LINES

```
+       if (cap_enter() < 0)
+               error("cap_enter: %s", pcap_strerror(errno));
        status = pcap_loop(pd, cnt, callback, pcap_userdata);
```

Figure 6: A two-line change adding capability mode to `tcpdump`: `cap_enter` is called prior to the main `libpcap` (packet capture) work loop. Access to global file system, IPC, and network namespaces is restricted.

- OPEN > LOCK > WORK
  - The historically buggy packet parser now runs with almost nothing

CAPSICUM – TCPDUMP IN TWO LINES

- But, “stdin” still readable (your keystrokes!)

```
+       if (lc_limitfd(STDIN_FILENO, CAP_FSTAT) < 0)
+               error("lc_limitfd: unable to limit STDIN_FILENO");
+       if (lc_limitfd(STDOUT_FILENO, CAP_FSTAT | CAP_SEEK | CAP_WRITE) < 0)
+               error("lc_limitfd: unable to limit STDOUT_FILENO");
+       if (lc_limitfd(STDERR_FILENO, CAP_FSTAT | CAP_SEEK | CAP_WRITE) < 0)
+               error("lc_limitfd: unable to limit STDERR_FILENO");
```

Figure 7: Using `lc_limitfd`, `tcpdump` can further narrow rights delegated by inherited file descriptors, such as limiting permitted operations on `STDIN` to `fstat`.

- stdin CAP_FSTAT can only read the file information
- stdout FSTAT | SEEK | WRITE can only print out
- stderr FSTAT | SEEK | WRITE can only print out

CAPSICUM – EXERCISE

- Let's sandbox an image decoder
  - What must it have before cap_enter()?
  - Which capabilities does each key need?
  - What might it try to open late …and break?

CAPSICUM – COST

- Entering the sandbox
  - cap_enter 1.22us
  - chroot 1.21us < setuid 1.39us
- Read on a capability
  - + 0.057us (+8.9%)
- A *new* sandbox process
  - libcapsicum: 1,509µs
  - vs. fork+exec 812µs (+86%)
  - Gzip: +2.4ms per run

Benchmark	Time/operation	Difference	% difference
dup	$1.061 \pm 0.000\mu s$	-	-
cap_new	$1.600 \pm 0.001\mu s$	$0.539 \pm 0.001\mu s$	$50.7\% \pm 0.08\%$
shmfd	$2.385 \pm 0.000\mu s$	-	-
cap_new_shmfd	$4.159 \pm 0.007\mu s$	$1.77 \pm 0.004\mu s$	$74.4\% \pm 0.181\%$
fstat_shmfd	$0.532 \pm 0.001\mu s$	-	-
fstat_cap_shmfd	$0.586 \pm 0.004\mu s$	$0.054 \pm 0.003\mu s$	$10.2\% \pm 0.506\%$
read_1	$0.640 \pm 0.000\mu s$	-	-
cap_read_1	$0.697 \pm 0.001\mu s$	$0.057 \pm 0.001\mu s$	$8.93\% \pm 0.143\%$
read_10000	$1.534 \pm 0.000\mu s$	-	-
cap_read_10000	$1.601 \pm 0.003\mu s$	$0.067 \pm 0.002\mu s$	$4.40\% \pm 0.139\%$
write	$0.576 \pm 0.000\mu s$	-	-
cap_write	$0.634 \pm 0.002\mu s$	$0.058 \pm 0.001\mu s$	$10.0\% \pm 0.241\%$
cap_enter	$1.220 \pm 0.000\mu s$	-	-
getuid	$0.353 \pm 0.001\mu s$	$-0.867 \pm 0.001\mu s$	$-71.0\% \pm 0.067\%$
chroot	$1.214 \pm 0.000\mu s$	$-0.006 \pm 0.000\mu s$	$-0.458\% \pm 0.023\%$
setuid	$1.390 \pm 0.001\mu s$	$0.170 \pm 0.001\mu s$	$14.0\% \pm 0.054\%$
fork	$268.934 \pm 0.319\mu s$	-	-
vfork	$44.548 \pm 0.067\mu s$	$-224.3 \pm 0.217\mu s$	$-83.4\% \pm 0.081\%$
pdfork	$259.359 \pm 0.118\mu s$	$-9.58 \pm 0.324\mu s$	$-3.56\% \pm 0.120\%$
pingpong	$309.387 \pm 1.588\mu s$	$40.5 \pm 1.08\mu s$	$15.0\% \pm 0.400\%$
fork_exec	$811.993 \pm 2.849\mu s$	-	-
vfork_exec	$585.830 \pm 1.635\mu s$	$-226.2 \pm 2.183\mu s$	$-27.9\% \pm 0.269\%$
pdfork_exec	$862.823 \pm 0.554\mu s$	$50.8 \pm 2.83\mu s$	$6.26\% \pm 0.348\%$
sandbox	$1509.258 \pm 3.016\mu s$	$697.3 \pm 2.78\mu s$	$85.9\% \pm 0.339\%$

RECAP OF SETUID AND CAPSICUM

	UNIX / setuid	Capsicum
Authority	from WHO you are (uid)	WHAT you hold (fd + rights)
To drop it	change identity: 3 uids 4 syscalls, OS-dependent	close or narrow a handle
Default	everything your user can do	nothing — after cap_enter
Policy	lives in file permission bits	in the program's own code

REVISIT – WHERE WE ARE

- Part I – Network/Internet security
  - The attacker is *outside* your machine
- Part II – Computer systems security
  - The attacker is *inside* your machine
  - Memory safety: how the attacker gets inside your machine (Buffer overflow)
  - OS security I and II: once inside, how can we *reduce* what the attacker can do?
    - SetUID demystified – drop the privileges that you don't need
    - Capsicum – Never hold those privileges in the first place
    - Flask – who decides the security rules
    - Syzkaller – kernel is the enforcer, and what if it contains security vulnerabilities?

FLASK – WHO DECIDES THE SECURITY RULES

WHO DECIDES THE ACCESS

	Who decides	Example
UNIX (DAC)	The file owner	chmod 777 my_secret.txt
Capsicum	Whoever holds the key	Pass the key to anyone
Flask (MAC)	The system policy	Enforced even against the owner

- Discretionary access control (DAC)
  - The owner of an object can give access to others
- Mandatory access control (MAC)
  - A system-wide policy decides; users (even root) cannot override it

NO SINGLE DEFINITION OF SECURITY

- Different environments need different policies
  - Military: users cleared for Secret cannot read Top Secret (MLS)
  - Hospital: only a patient's doctors can read the chart (RBAC)
  - Web server: the server process can read web files only (TE)

- Prior OSes hard-coded one policy (or none)

- Flask: one OS mechanism that supports many policies
  - Change the policy, not the kernel

FLASK – POLICY FLEXIBILITY

- What does the OS need to support many policies?
  - Control the propagation of access rights
  - Fine-grained access control on low-level operations
  - Revocation of the access rights already granted

- Revocation is the most difficult one
  - Once granted, permissions migrate throughout the system

WHY REVOCATION IS DIFFICULT

Try this in your shell

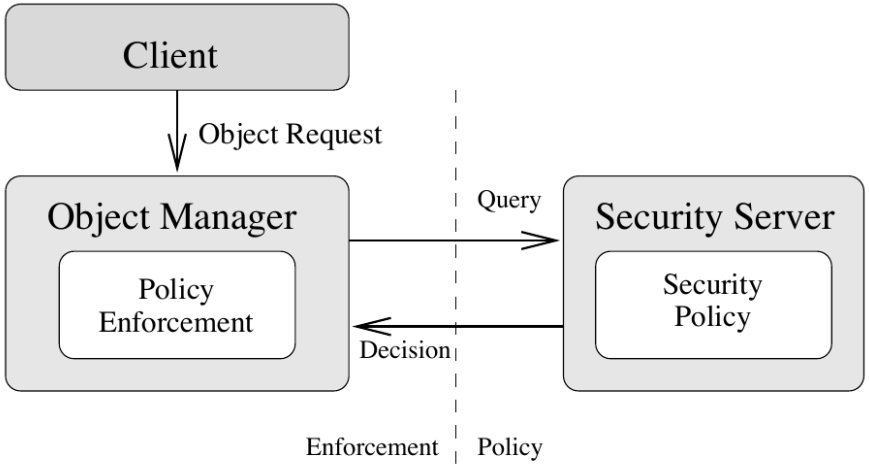
```
$ exec 3>>notes.txt        # open for writing
$ chmod 444 notes.txt      # remove the write permission
$ echo "still here" >&3    # write through the open fd
$ cat notes.txt
still here
```

- UNIX checks the permission only at open()
  - The result is cached in the file descriptor (migrated permission)
  - Also in page tables, IPC connections, operations in progress

FLASK VS. CAPABILITIES

- Flask (1999): capabilities are poorly suited to policy flexibility
  - The holder of a capability decides who gets a copy, not the policy
- Capsicum (2010): it is the right design
  - The program knows what each component needs
  - The policy lives in the code
- Poll: who should decide what the browser renderer can access?
  - (a) Browser developers (b) OS administrator (c) Both

FLASK – ARCHITECTURE



- Object manager
  - Kernel, file server, process manager
  - Enforces the decisions
  - No policy logic
- Security server
  - Makes the decisions
  - The only place the policy lives
- Change the policy, not the object managers

FLASK – LABELS

- Every subject (process) and object (file, socket, …) has a label
  - Security context, e.g., user:role:type in SELinux
  - Policy rules are mostly written between types

- Security ID (SID): a 32-bit number mapped to a context
  - Object managers only handle SIDs; they do not parse labels
  - A new object gets its label from the security server

- Examples

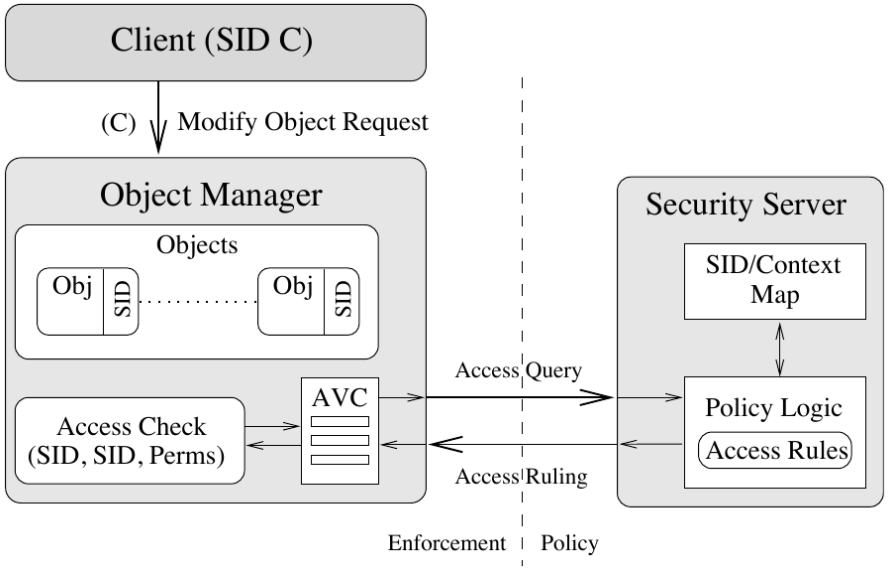
```
/etc/shadow  (file)       system_u:object_r:shadow_t
passwd       (process)    staff_u:staff_r:passwd_t
```

FLASK – AN EXAMPLE

- passwd (passwd_t) opens /etc/shadow (shadow_t) for write
  - The object manager asks: (passwd_t, shadow_t, write)?
  - The security server checks the policy
    - `allow passwd_t shadow_t : file { read write };`
  - Allowed; the object manager opens the file and caches the decision

- The web server (httpd_t) reads /etc/shadow
  - No rule for (httpd_t, shadow_t), so it is denied
  - Even when the web server runs as root

FLASK – ACCESS DECISION



- Client requests to modify an object
- Object manager checks its cache (AVC)
  - (client SID, object SID, perms)
- On a miss, ask the security server
- The answer (an access vector) is cached
  - All permissions for that SID pair

FLASK – REVOCATION

- When the policy changes, the security server notifies the object managers
  - Each object manager updates its cache
  - And revokes the migrated permissions (e.g., thread and memory state)
  - Then it tells the security server that the change is done
- The change takes effect only after all object managers finish
- Object managers must finish in a timely manner
  - Untrusted software should not delay a revocation

FLASK – COST

- Null IPC round trip
  - +2% (hint), +7% (cache)
  - +511% (ask the server)
- Compiling with make
  - 681,335 decisions
  - Only 39 reached the server
  - +4.5% (disk), +11% (memfs)
- Code size
  - +4,575 lines (< 8%)

		Flask			
	Fluke	using hint	using cache	calling trivSS	calling realSS
''Null''	13.5 μs	13.8 μs +2%	14.4 μs +7%	43.4 μs +221%	82.5 μs +511%

		Resolution		
Object Manager	Total queries	using hint	using cache	calling SS
Kernel	603735	175585	428121	29
FFS	76708	N/A	76700	8
PM	892	N/A	890	2

FROM FLASK TO SELINUX

- NSA ported the Flask architecture to Linux: SELinux (2000)
  - Linux Security Modules (LSM) hooks since Linux 2.6
- Android uses SELinux in enforcing mode since Android 5.0
- FreeBSD and macOS: TrustedBSD MAC framework
  - macOS Seatbelt sandbox is built on it
  - Recall the Chromium table: Seatbelt (MAC), SELinux (MAC)

SELINUX – TYPE ENFORCEMENT

```
/etc/shadow           shadow_t
passwd process        passwd_t
web server process    httpd_t

allow passwd_t shadow_t : file { read write };
```

- No rule for httpd_t: denied, even if the web server runs as root
- Recall: a buffer overflow in a setuid-root binary = root
  - With type enforcement, root in the wrong domain is still limited

SELINUX – DISCUSSION

- Why do many people turn SELinux off? (setenforce 0)
- Flask makes the policy replaceable, but not easy to write
  - *"We have yet to explore the development of a more expressive policy specification language"*
- Who pays for a missing rule? Who pays for a too-broad rule?

RECAP OF DAC, CAPSICUM, AND FLASK

	UNIX DAC	Capsicum	Flask / SELinux
Who decides	Owner	Key holder	Policy
Policy lives in	Permission bits	Program code	Security server
Root can bypass	Yes	N/A	No
Revoke a right	No (fd keeps it)	Close the key	Yes, system-wide

SYZKALLER – WHO CHECKS THE REFEREE?

THE KERNEL IS THE ENFORCER

- setuid, Capsicum, and SELinux checks all run inside the kernel
- A memory bug in the kernel can let the attacker
  - Overwrite their credentials (uid = 0)
  - Turn off SELinux enforcement
  - Escape the Capsicum sandbox
- The kernel is written in C, tens of millions of lines

FUZZING PRELIM

- An automated testing technique
  - Run a program with many inputs until it crashes
- History
  - 1990s: random inputs
  - 2000s: mutate valid inputs (bit flips, insertion, deletion)
  - 2010s: coverage-guided fuzzing (e.g., AFL)
- Coverage-guided fuzzing
  - Keep the inputs that reach new code, and mutate them again

KERNEL FUZZING

- A crash takes down the whole machine
  - Run the target kernel inside VMs
- The input is a sequence of system calls, not a file
  - read() needs an fd returned by open()
- Kernel state persists across calls
- Threads, interrupts, and devices make bugs hard to reproduce

SYZKALLER – PROGRAMS AS INPUTS

```
r0 = open(&(0x7f0000000000)="./file0", 0x3, 0x9)
read(r0, &(0x7f0000000000), 42)
close(r0)
```

- An input is a program: a sequence of system calls
- r0 is a resource: the fd from open() is passed to read()
  - Random bytes rarely produce a valid fd
- Syzkaller generates, mutates, and minimizes these programs

SYZKALLER – SYSTEM CALL DESCRIPTIONS

```
open(file filename, flags flags[open_flags], mode flags[open_mode]) fd
read(fd fd, buf buffer[out], count len[buf])
close(fd fd)
```

- Describe the arguments of every system call (syzlang)
  - flags[...]: one of these constants; len[buf]: size of buf
  - fd: connects one call's output to another call's input
- Written by hand, one file per subsystem (sys/linux/*.txt)
  - Interfaces without descriptions are not tested

SYZKALLER – COVERAGE

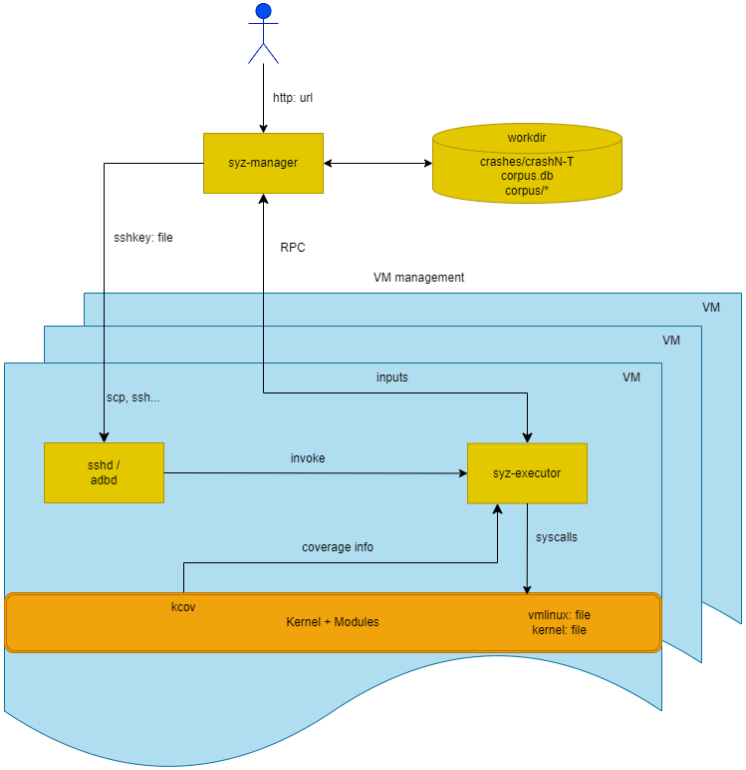
- The compiler inserts a probe at each branch of the kernel
- KCOV reports which probes a program hits
- A program that reaches new branches is kept in the corpus
- Same idea as AFL, but the coverage comes from the kernel

SYZKALLER – SANITIZERS

- Many memory bugs do not crash the kernel
- The target kernel is built with sanitizers
  - KASAN: AddressSanitizer for the kernel (recall the ASan talk)
  - UBSAN, KMSAN, lockdep, ...
- An example syzbot report

```
BUG: KASAN: slab-use-after-free in kobject_get_path+0x91/0x2f0
Read of size 8 at addr ffff88802a29ee18 by task kworker/1:2/1654
Call Trace:
 kasan_report+0x117/0x150 mm/kasan/report.c:595
 kobject_get_path+0x91/0x2f0 lib/kobject.c:158
 drm_sysfs_hotplug_event+0x10e/0x150 drivers/gpu/drm/drm_sysfs.c:451
```

SYZKALLER – ARCHITECTURE



- syz-manager (host)
  - Generates and mutates programs
  - Stores the corpus and crashes
- syz-executor (in each VM)
  - Runs the programs
  - Reports coverage via KCOV
- On a crash
  - Save the log and restart the VM
  - syz-repro finds a minimal reproducer

SYZBOT – CONTINUOUS FUZZING

- Fuzzes the main Linux branches continuously
- Reports bugs to the kernel mailing lists automatically
  - Crash report, reproducer, and a Reported-by: syzbot tag
- Tracks the fix until it reaches all tested branches
- Upstream Linux (as of 10/21/2026)
  - 7,571 bugs fixed, 1,332 bugs open

SYZKALLER – DISCUSSION

- 1,332 open bugs: finding is automated, but fixing is not
  - Who fixes a bug in an unmaintained driver?
- A missing permission check does not crash
  - e.g., a missing SELinux hook: no KASAN report, no crash
  - Fuzzers find crashes, not wrong policies
- System calls without descriptions are not tested

RECAP OF PART II

	Question	Papers
Memory safety	How does the attacker get in?	StackGuard, BROP, ASan
OS security I	What can they do inside?	SetUID, Capsicum
OS security II	Who makes the rules?	Flask
	Who checks the kernel?	Syzkaller

- Next (Mon. 10/26): Part III – Isolation, in person (OWEN 106)
  - Critique: Software-based fault isolation OR Haven

Thank You!

Sanghyun Hong

https://true-lab.ai/courses/Sec-Grad/current



